

Python Application Programming

Anitya Gupta

Stride

Name = “The BodyGuard”

T	h	e		B	o	d	y	G	u	a	r	d
0	1	2	3	4	5	6	7	8	9	10	11	12

Name[::2] = “TeBdGad”

Concatenate Strings

```
[17]: # Concatenate two strings

statement = name + " is the best album"
statement
```

```
[17]: 'The BodyGuard is the best album'
```

To replicate values of a string we simply multiply the string by the number of times we would like to replicate it. In this case, the number is three. The result is a new string, and this new string consists of three copies of the original string:

```
[18]: # Print the string for 3 times

3 * "The BodyGuard"
```

```
[18]: 'The BodyGuardThe BodyGuardThe BodyGuard'
```

You can create a new string by setting it to the original variable. Concatenated with a new string, the result is a new string that changes from The BodyGuard to "The BodyGuard is the best album".

```
[19]: # Concatenate strings

name = "The BodyGuard"
name = name + " is the best album"
name
```

Different Escape Sequence

[20]: *# New Line escape sequence*

```
print(" The BodyGuard\n is the best album" )
```

The BodyGuard
is the best album

Similarly, back slash "t" represents a tab:

[21]: *# Tab escape sequence*

```
print(" The BodyGuard \t is the best album" )
```

The BodyGuard is the best album

If you want to place a back slash in your string, use a double back slash:

[22]: *# Include back slash in string*

```
print(" The BodyGuard \\ is the best album" )
```

The BodyGuard \ is the best album

String Manipulation Operators

```
# Convert all the characters in string to upper case
```

```
a = "Thriller is the sixth studio album"
```

```
print("before upper:", a)
```

```
b = a.upper()
```

```
print("After upper:", b)
```

```
before upper: Thriller is the sixth studio album
```

```
After upper: THRILLER IS THE SIXTH STUDIO ALBUM
```

Replace

The method `replace` replaces a segment of the string, i.e. a substring with a new string. We input the part of the string we would like to change. The second argument is what we would like to exchange the segment with, and the result is a new string with the segment changed:

```
[25]: # Replace the old substring with the new target substring is the segment has been found in the string
```

```
a = "The BodyGuard is the best album"  
b = a.replace('BodyGuard', 'Janet')  
b
```

```
[25]: 'The Janet is the best album'
```

Find

The method `find` finds a sub-string. The argument is the substring you would like to find, and the output is the first index of the sequence. We can find the sub-string `he` or `Guard`.

Name = "The BodyGuard"

T	h	e		B	o	d	y	G	u	a	r	d
0	1	2	3	4	5	6	7	8	9	10	11	12

Name.find('he'):1

Name.find('Guard'):8

```
# Find the substring in the string. Only the index of the first element of substring in string will be the output  
  
name = "The BodyGuard"  
name.find('he')
```

```
1
```

Split

The method `Split` splits the string at the specified separator, and returns a list.

Syntax

```
string.split(separator, maxsplit)
```

Parameters

- separator (optional): This is the delimiter at which the string will be split. If not provided, the default separator is any whitespace.
- maxsplit (optional): This specifies the maximum number of splits to perform. If not provided, there is no limit on the number of splits.

Return Value:

The method returns a list of substrings.

```
29]: #Split the substring into list
      name = "The BodyGuard"
      split_string = (name.split())
      split_string
```

```
29]: ['The', 'BodyGuard']
```


RegEX

This RegEx module provides several functions for working with regular expressions, including `search`, `split`, `findall`, and `sub`.

Python provides a built-in module called `re`, which allows you to work with regular expressions. First, import the `re` module

```
[30]: import re
```

The `search()` function searches for specified patterns within a string. Here is an example that explains how to use the `search()` function to search for the word "Body" in the string "The BodyGuard is the best".

```
[31]: s1 = "The BodyGuard is the best album"

# Define the pattern to search for
pattern = r"Body"

# Use the search() function to search for the pattern in the string
result = re.search(pattern, s1)

# Check if a match was found
if result:
    print("Match found!")
else:
    print("Match not found.")
```

Match found!

Special Sequence of RegEX

Special Sequence	Meaning	Example
\d	Matches any digit character (0-9)	"123" matches "\d\d\d"
\D	Matches any non-digit character	"hello" matches "\D\D\D\D\D"
\w	Matches any word character (a-z, A-Z, 0-9, and _)	"hello_world" matches "\w\w\w\w\w\w\w\w\w\w\w"
\W	Matches any non-word character	"@#\$%" matches "\W\W\W\W"
\s	Matches any whitespace character (space, tab, newline, etc.)	"hello world" matches "\w\w\w\w\w\s\w\w\w\w\w"
\S	Matches any non-whitespace character	"hello_world" matches "\S\S\S\S\S\S\S\S\S"
\b	Matches the boundary between a word character and a non-word character	"cat" matches "\bcat\b" in "The cat sat on the mat"
\B	Matches any position that is not a word boundary	"cat" matches "\Bcat\B" in "category" but not in "The cat sat on the mat"

match.group()

```
32]: pattern = r"\d\d\d\d\d\d\d\d\d\d" # Matches any ten consecutive digits
text = "My Phone number is 1234567890"
match = re.search(pattern, text)

if match:
    print("Phone number found:", match.group())
else:
    print("No match")
```

Phone number found: 1234567890

The `match.group()` method is used in Python's `re` module to retrieve the part of the string where the regular expression pattern matched. Here's a detailed explanation:

Purpose

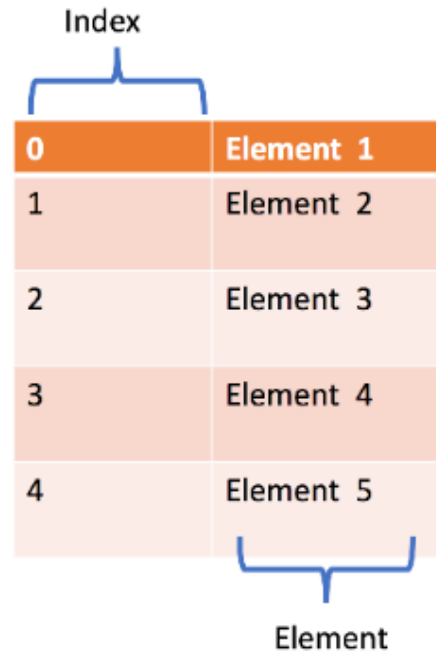
- Extract Matched Text: `match.group()` returns the exact substring that matched the pattern.

Usage

- When you use functions like `re.search()` or `re.match()`, they return a match object if the pattern is found. You can then use `match.group()` to get the matched text.

Here `match.group()` retrieves the substring 1234567890 from the text, which is the part that matched the pattern.

Lists



A diagram illustrating a list structure. It consists of a table with two columns. The first column is labeled 'Index' with a blue bracket above it. The second column is labeled 'Element' with a blue bracket below it. The table contains five rows of data. The first row has an orange background, while the others are light pink. The elements are labeled 'Element 1' through 'Element 5'.

Index	Element
0	Element 1
1	Element 2
2	Element 3
3	Element 4
4	Element 5

[Element 1 , Element 2 , Element 3 , Element 4, Element 5]

Index 0 1 2 3 4

List – Slicing

```
[4]: # Sample List
```

```
L = ["Michael Jackson", 10.1, 1982, "MJ", 1]  
L
```

```
[4]: ['Michael Jackson', 10.1, 1982, 'MJ', 1]
```

L = ["Michael Jackson", 10.1, 1982, "MJ", 1]

0	1	2	3	4
---	---	---	---	---

```
[5]: # List slicing
```

```
L[3:5]
```

```
[5]: ['MJ', 1]
```

We can use the method `extend` to add new elements to the list:

Operations

We can use the method `extend` to add new elements to the list:

```
[6]: # Use extend to add elements to list
```

```
L = [ "Michael Jackson", 10.2]  
L.extend(['pop', 10])  
L
```

```
[6]: ['Michael Jackson', 10.2, 'pop', 10]
```

Another similar method is `append`. If we apply `append` instead of `extend`, we add one element to the list:

```
[7]: # Use append to add elements to list
```

```
L = [ "Michael Jackson", 10.2]  
L.append(['pop', 10])  
L
```

```
[7]: ['Michael Jackson', 10.2, ['pop', 10]]
```

Are Lists Mutable!

As lists are mutable, we can change them. For example, we can change the first element as follows:

```
[10]: # Change the element based on the index
```

```
A = ["disco", 10, 1.2]
print('Before change:', A)
A[0] = 'hard rock'
print('After change:', A)
```

Before change: ['disco', 10, 1.2]

After change: ['hard rock', 10, 1.2]

We can also delete an element of a list using the `del` command:

```
[11]: # Delete the element based on the index
```

```
print('Before change:', A)
del(A[0])
print('After change:', A)
```

Before change: ['hard rock', 10, 1.2]

After change: [10, 1.2]